OCOM5300M Artificial Intelligence Project Cross Attention LSTM Decoder

In [1]:
```python
# Custom Classes
from captions import Captions
from vocabulary import Vocabulary
from dataset import ROCOv2_Subset
from pipeline import AdditiveCrossAttention, Decoder, DecoderModel

# Deep learning Functionality
import torch  # PyTorch library
import torch.nn as nn  # PyTorch neural network module
from torch.utils.data import Dataset, DataLoader  # creating custom data containers, batching, shuffling and data loading capabilities
import torch.nn.functional as F  # PyTorch functional interface e.g., activation functions, loss calculations
import torch.optim as optim  # optimization algorithms for training neural networks

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
from collections import Counter  # counting occurrences, e.g., word frequencies

# Visualization
import matplotlib.pyplot as plt  # plotting and visualizing results
import matplotlib.gridspec as gridspec #creating grid-based layouts of subplots
from tqdm import tqdm # progress bars

# Utilities
import os  # file and directory operations
import time  # timing operations
import re #regular expressions
import math # common mathematical functions
from pathlib import Path # file and directory operations
```

In [2]:
```python
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```python
# device
device = torch.device('cpu')
```

In [4]:
```
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [5]:
```
# set random seed to 42
seed = 42
```

In [6]:
```
# seed set to ensure reproducibility of results
torch.manual_seed(seed)
```

Out[6]:
```
<torch._C.Generator at 0x1f0a70418b0>
```

Data Preparation

In [7]:
```
# training captions filename
training_captions_filename = "ROCOv2/train_captions.csv"
```

In [8]:
```
# instantiate training captions object
training_captions = Captions()
```

In [9]:
```
# load training captions
training_caption_data = training_captions.load_captions(training_captions_filename)
```

In [10]:
```
# clean training captions
training_captions.clean_captions();
```

In [11]:
```
# confirm number of cleaned training captions
len(training_caption_data)
```

Out[11]:
```
59958
```

In [12]:
```
# maximum caption length
maximum_caption_length = (training_captions.get_caption_length_percentile(99))
```

In [13]:
```
# validation captions filename
valid_captions_filename = "ROCOv2/valid_captions.csv"
```

In [14]:
```
# instantiate validation captions object
valid_captions = Captions()
```

In [15]:
```
# load validation captions
valid_caption_data = valid_captions.load_captions(valid_captions_filename)
```

In [16]:
```
# clean validation captions
valid_captions.clean_captions();
```

In [17]:
```
# confirm number of cleaned validation captions
len(valid_caption_data)
```

Out[17]:
```
9904
```

In [18]:
```
# instantiate Vocabulary object
vocabulary = Vocabulary()
```

In [19]:
```
# reload vocabulary
vocabulary = Vocabulary.load("ROCOv2/vocabulary.pt")
```

In [20]:
```
# confirm number of tokens in vocabulary, including special tokens
len(vocabulary)
```

Out[20]:
```
17602
```

In [21]:
```
# set batch size to 32 for data loading
batch_size = 32
```

In [22]:
```
# feature files folder for training images
training_feature_folder = (r"ROCOv2\train_images\train\pt")
```

In [23]:
```
# create training dataset
train_dataset = ROCOv2_Subset(
    training_caption_data,
    training_feature_folder,
    vocabulary,
```

```
    maximum_caption_length,
)
```

In [24]:
```
# training set dataloader
train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True # set shuffle = true for the training set data loader
    )
```

In [25]:
```
# feature files folder for training images
valid_feature_folder = (r"ROCOv2\valid_images\valid\pt")
```

In [26]:
```
# create validation dataset
valid_dataset = ROCOv2_Subset(
    valid_caption_data,
    valid_feature_folder,
    vocabulary,
    maximum_caption_length,
)
```

In [27]:
```
# validation set dataloader
valid_loader = DataLoader(
        valid_dataset,
        batch_size=batch_size,
        shuffle=False # set shuffle = false
)
```

Decoder

In [28]:
```
# set hyperparameter, i.e., the dimensionality of the word embedding vectors in the model
embed_size = 256
```

In [29]:
```
# set hyperparameter, i.e., dimensionality of the hidden state in LSTM layers
hidden_size = 512
```

In [30]:
```
# set hyperparameter, i.e., the number of stacked recurrent layers in the neural network architecture
number_of_layers = 1
```

In [31]:
```
# set encoder feature dimension
encoder_dimension = 2048
```

In [32]:
```
# set attention dimension, i.e., the size of the intermediate feature space used to calculate attention scores
# for relevancy of each image region when the decoder is generating the next word
attention_dimension = 512
```

In [33]:
```
# specify the saved best-performing model file, i.e., based on the epoch with the lowest validation loss
selected_model = 'cross_attention_lstm_best.pth'
```

In [34]:
```
# initial step size when updating the model weights
learning_rate = 0.0015
```

In [35]:
```
# size of penalty to be applied large model weights to help reduce overfitting.
weight_decay = 1e-4
```

In [36]:
```
# maximum gradient size allowed before clipping to help prevent unstable weight updates
maximum_gradient_size = 1.0
```

In [37]:
```
#number of epochs
number_of_epochs = 20
```

In [38]:
```
# early stopping patience
early_stopping_patience = 6
```

In [39]:
```
# dropout, i.e., 30% of the selected neuron activations are randomly set to zero on each forward pass.
# This forces the model not to rely too heavily on particular neurons
# and can improve generalisation to unseen data to prevent overfitting
dropout = 0.3
```

In [40]:
```
# select visual gate type: scalar, vector or none
visual_gate_type = "none"
```

In [41]:
```
# numerical index assigned to the <pad> token in the vocabulary
pad_idx = vocabulary("<pad>")
```

In [42]:
```python
# create decoder model
decoder = DecoderModel(
    vocab_size=len(vocabulary),
    pad_idx=pad_idx,
    encoder_dim=encoder_dimension,
    embed_size=embed_size,
    hidden_size=hidden_size,
    attention_dim=attention_dimension,
    num_layers=number_of_layers,
    max_seq_length=maximum_caption_length,
    dropout=dropout,
    visual_gate_type=visual_gate_type
)
```

In [43]:
```python
# train model
model_statistics = decoder.train_model(
    train_loader=train_loader,
    val_loader=valid_loader,
    num_epochs=number_of_epochs,
    learning_rate=learning_rate,
    weight_decay=weight_decay,
    patience=early_stopping_patience,
    clip_grad_norm=maximum_gradient_size,
    checkpoint_path=selected_model,
    device=device
)
```

```
Cross-attention LSTM training 1/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 1/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [1/20] - 7111.18s
Training loss: 5.4005 | Training perplexity: 221.5135
Validation loss: 4.6264 | Validation perplexity: 102.1453
Learning rate: 0.001500
Overfitting gap: -0.7741
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 2/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 2/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [2/20] - 6184.10s
Training loss: 4.5500 | Training perplexity: 94.6304
Validation loss: 4.3534 | Validation perplexity: 77.7387
Learning rate: 0.001500
Overfitting gap: -0.1966
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 3/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 3/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [3/20] - 7368.13s
Training loss: 4.3664 | Training perplexity: 78.7627
Validation loss: 4.2516 | Validation perplexity: 70.2182
Learning rate: 0.001500
Overfitting gap: -0.1148
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 4/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 4/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [4/20] - 6303.71s
Training loss: 4.2795 | Training perplexity: 72.2028
Validation loss: 4.1982 | Validation perplexity: 66.5682
Learning rate: 0.001500
Overfitting gap: -0.0813
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 5/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 5/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [5/20] - 6019.40s
Training loss: 4.2274 | Training perplexity: 68.5382
Validation loss: 4.1637 | Validation perplexity: 64.3114
Learning rate: 0.001500
Overfitting gap: -0.0637
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 6/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 6/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [6/20] - 5895.16s
Training loss: 4.1909 | Training perplexity: 66.0817
Validation loss: 4.1332 | Validation perplexity: 62.3781
Learning rate: 0.001500
Overfitting gap: -0.0577
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 7/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 7/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [7/20] - 5908.85s
Training loss: 4.1650 | Training perplexity: 64.3902
Validation loss: 4.1164 | Validation perplexity: 61.3369
Learning rate: 0.001500
Overfitting gap: -0.0486
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 8/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 8/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [8/20] - 5884.60s
Training loss: 4.1455 | Training perplexity: 63.1501
Validation loss: 4.1053 | Validation perplexity: 60.6592
Learning rate: 0.001500
Overfitting gap: -0.0402
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 9/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 9/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [9/20] - 5881.04s
Training loss: 4.1282 | Training perplexity: 62.0684
Validation loss: 4.0897 | Validation perplexity: 59.7237
Learning rate: 0.001500
Overfitting gap: -0.0385
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 10/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 10/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [10/20] - 5911.88s
Training loss: 4.1118 | Training perplexity: 61.0593
Validation loss: 4.0717 | Validation perplexity: 58.6544
Learning rate: 0.001500
Overfitting gap: -0.0402
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 11/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 11/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [11/20] - 5868.80s
Training loss: 4.0994 | Training perplexity: 60.3020
Validation loss: 4.0673 | Validation perplexity: 58.4019
Learning rate: 0.001500
Overfitting gap: -0.0320
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 12/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 12/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [12/20] - 5839.13s
Training loss: 4.0871 | Training perplexity: 59.5670
Validation loss: 4.0669 | Validation perplexity: 58.3737
Learning rate: 0.001500
Overfitting gap: -0.0202
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 13/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 13/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [13/20] - 5861.82s
Training loss: 4.0755 | Training perplexity: 58.8808
Validation loss: 4.0481 | Validation perplexity: 57.2888
Learning rate: 0.001500
Overfitting gap: -0.0274
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 14/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 14/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [14/20] - 5911.64s
Training loss: 4.0657 | Training perplexity: 58.3029
Validation loss: 4.0452 | Validation perplexity: 57.1244
Learning rate: 0.001500
Overfitting gap: -0.0204
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 15/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 15/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [15/20] - 5930.82s
Training loss: 4.0562 | Training perplexity: 57.7571
Validation loss: 4.0288 | Validation perplexity: 56.1938
Learning rate: 0.001500
Overfitting gap: -0.0274
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 16/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 16/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [16/20] - 5932.08s
Training loss: 4.0481 | Training perplexity: 57.2913
Validation loss: 4.0365 | Validation perplexity: 56.6298
Learning rate: 0.001500
Overfitting gap: -0.0116
No validation improvement for 1 epoch(s).
Cross-attention LSTM training 17/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 17/20:   0%|          | 0/310 [00:00<?, ?it/s]
```

```
Epoch [17/20] - 5920.94s
Training loss: 4.0397 | Training perplexity: 56.8077
Validation loss: 4.0247 | Validation perplexity: 55.9636
Learning rate: 0.001500
Overfitting gap: -0.0150
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 18/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 18/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [18/20] - 5958.41s
Training loss: 4.0328 | Training perplexity: 56.4206
Validation loss: 4.0202 | Validation perplexity: 55.7147
Learning rate: 0.001500
Overfitting gap: -0.0126
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 19/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 19/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [19/20] - 5937.06s
Training loss: 4.0254 | Training perplexity: 56.0020
Validation loss: 4.0138 | Validation perplexity: 55.3550
Learning rate: 0.001500
Overfitting gap: -0.0116
Validation loss improved. Saved checkpoint to 'cross_attention_lstm_best.pth'.
Cross-attention LSTM training 20/20:   0%|          | 0/1874 [00:00<?, ?it/s]
Cross-attention LSTM validation 20/20:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [20/20] - 5956.05s
Training loss: 4.0207 | Training perplexity: 55.7391
Validation loss: 4.0146 | Validation perplexity: 55.4026
Learning rate: 0.001500
Overfitting gap: -0.0061
No validation improvement for 1 epoch(s).
Best validation loss: 4.0138 at epoch 19.
```

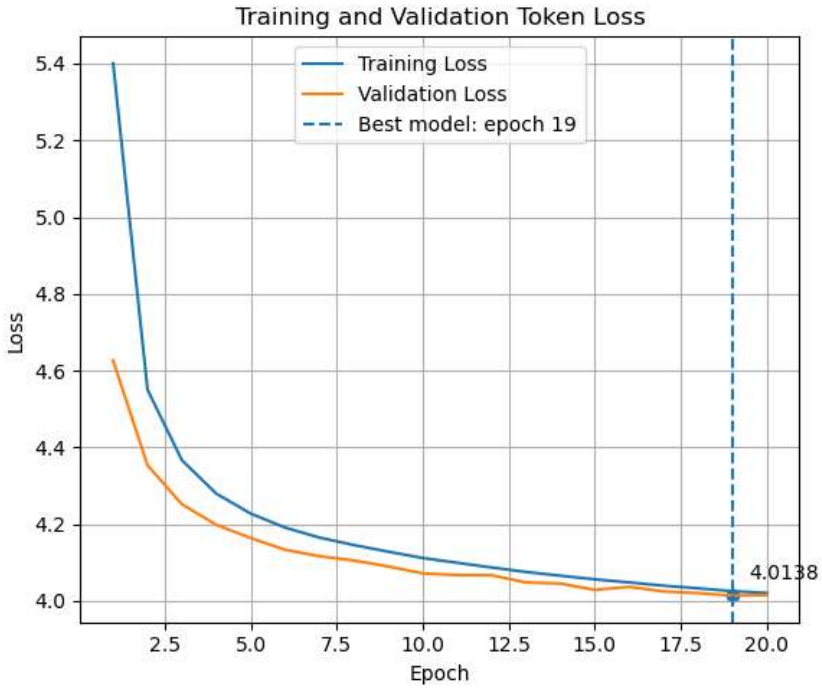
Diagnostics

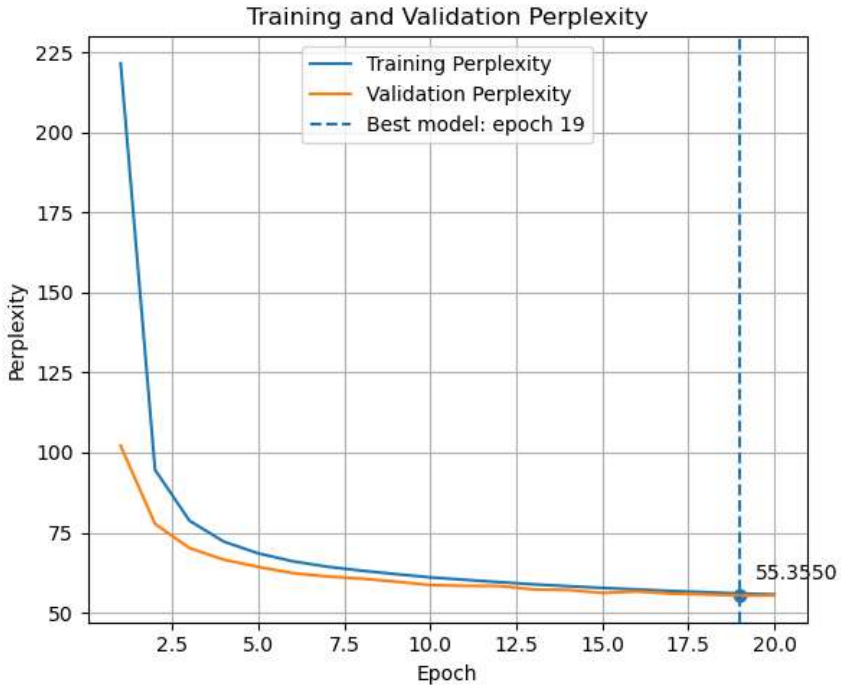
In [44]:
```
# display model's epoch statistics
model_statistics
```

Out[44]:

	epoch	training loss	validation loss	training perplexity	validation perplexity	learning rate	overfitting
0	1	5.400484	4.626396	221.513529	102.145251	0.0015	-0.774088
1	2	4.549978	4.353354	94.630355	77.738737	0.0015	-0.196625
2	3	4.366440	4.251607	78.762706	70.218179	0.0015	-0.114832
3	4	4.279478	4.198227	72.202751	66.568183	0.0015	-0.081251
4	5	4.227392	4.163737	68.538245	64.311410	0.0015	-0.063655
5	6	4.190892	4.133214	66.081688	62.378098	0.0015	-0.057677
6	7	4.164961	4.116382	64.390188	61.336934	0.0015	-0.048579
7	8	4.145514	4.105271	63.150051	60.659201	0.0015	-0.040242
8	9	4.128237	4.089729	62.068380	59.723685	0.0015	-0.038508
9	10	4.111845	4.071663	61.059271	58.654404	0.0015	-0.040182
10	11	4.099365	4.067349	60.301970	58.401942	0.0015	-0.032016
11	12	4.087102	4.066866	59.567015	58.373725	0.0015	-0.020236
12	13	4.075515	4.048104	58.880805	57.288757	0.0015	-0.027411
13	14	4.065652	4.045231	58.302908	57.124378	0.0015	-0.020421
14	15	4.056247	4.028806	57.757149	56.193763	0.0015	-0.027441
15	16	4.048149	4.036536	57.291309	56.629846	0.0015	-0.011613
16	17	4.039671	4.024702	56.807655	55.963616	0.0015	-0.014969
17	18	4.032835	4.020244	56.420631	55.714722	0.0015	-0.012590
18	19	4.025388	4.013766	56.002013	55.354962	0.0015	-0.011621
19	20	4.020682	4.014627	55.739080	55.402620	0.0015	-0.006055

In [45]:
```
# display performance graphs
decoder.plot_model_performance()
```


Training and Validation Token Loss
Training Loss
Validation Loss
Best model: epoch 19
Loss
Epoch
4.0138
5.4
5.2
5.0
4.8
4.6
4.4
4.2
4.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Training and Validation Perplexity
Training Perplexity
Validation Perplexity
Best model: epoch 19
Perplexity
Epoch
55.3550
225
200
175
150
125
100
75
50
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

Out[45]:

```
DecoderModel(
  (embedding): Embedding(17602, 256, padding_idx=0)
  (cross_attention): AdditiveCrossAttention(
    (visual_projection): Linear(in_features=2048, out_features=512, bias=True)
    (hidden_projection): Linear(in_features=512, out_features=512, bias=True)
    (attention_score): Linear(in_features=512, out_features=1, bias=False)
  )
  (lstm_cells): ModuleList(
    (0): LSTMCell(2304, 512)
  )
  (initial_hidden): ModuleList(
    (0): Linear(in_features=2048, out_features=512, bias=True)
  )
  (initial_cell): ModuleList(
    (0): Linear(in_features=2048, out_features=512, bias=True)
  )
  (visual_gate): Linear(in_features=512, out_features=1, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (hidden_to_embed): Linear(in_features=512, out_features=256, bias=True)
  (output_layer): Linear(in_features=256, out_features=17602, bias=False)
)
```

In [46]:

```python
def plot_gate_diagnostics(decoder, data_loader, device):
    """
    Calculate and plot the mean visual gate value at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: gate diagnostics by caption timestep
    """

    try:

        # calculate mean visual gate values by caption position
        gate_diagnostics = decoder.calculate_gate_mean_by_timestep(
            data_loader,
            device
        )
```

```
        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual gate by caption position
        plt.plot(
            gate_diagnostics["Timestep"],
            gate_diagnostics["Mean Gate"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Gate")
        plt.title("Mean Visual Gate by Caption Position")
        plt.ylim(0, 1)
        plt.grid(True)

        # display plot
        plt.show()

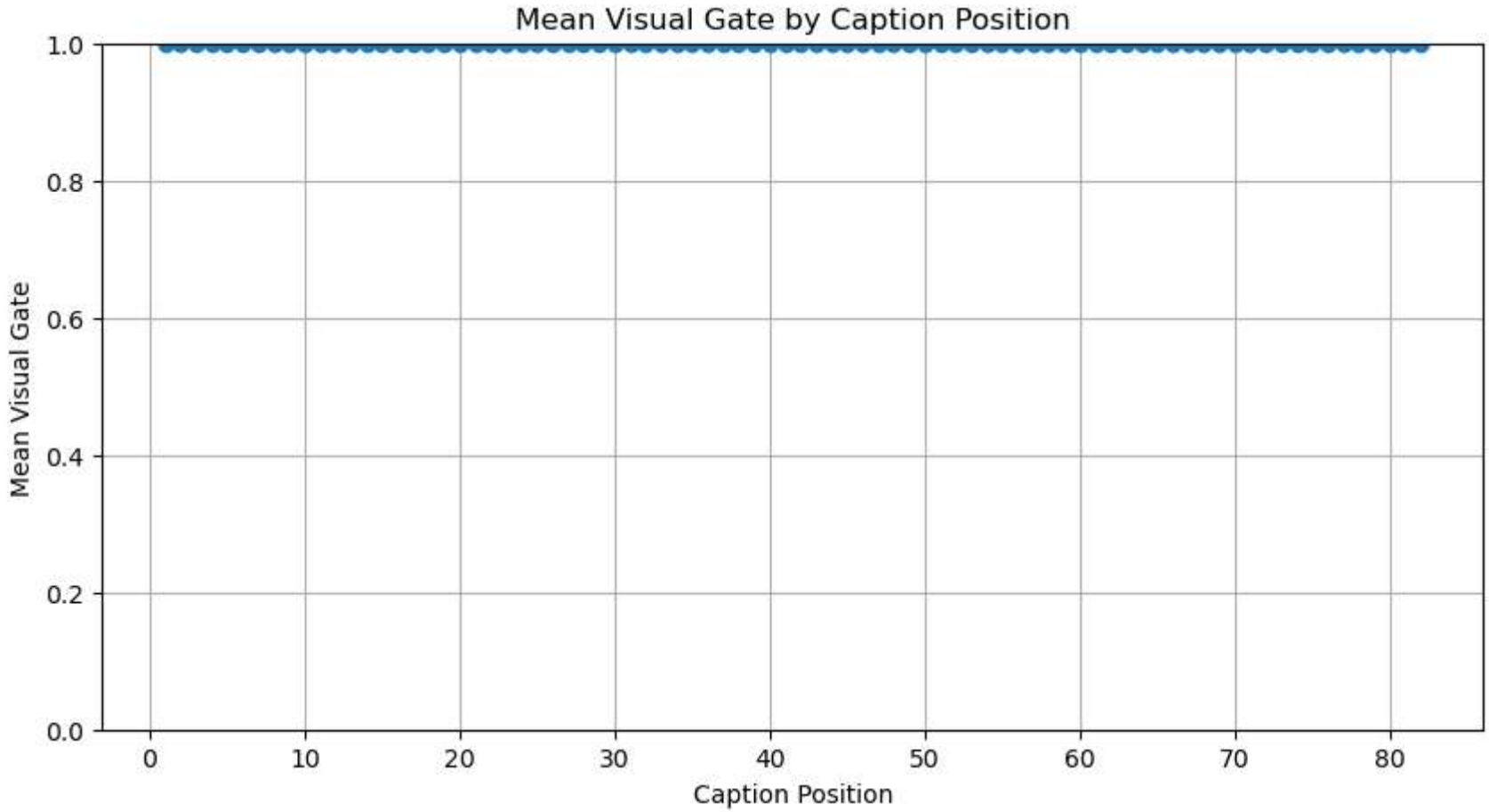
        return gate_diagnostics

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [47]:

```
# the gate is a learned control value that regulates how much visual information from the image is allowed into the
decoder at each caption timestep.
# calculate mean visual gate value by caption timestep
gate_diagnostics = plot_gate_diagnostics(decoder, valid_loader, device)
```

```
Gate diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [48]:
```
# display gate diagnostics
display(gate_diagnostics)
```

	Timestep	Mean Gate	Valid Captions
0	1	1.0	9904.0
1	2	1.0	9903.0
2	3	1.0	9898.0
3	4	1.0	9826.0
4	5	1.0	9715.0
...	...	...	...
77	78	1.0	186.0
78	79	1.0	177.0
79	80	1.0	170.0
80	81	1.0	162.0
81	82	1.0	152.0

82 rows × 3 columns

In [49]:
```python
def plot_visual_context_diagnostics(decoder, data_loader, device):
    """
    Calculate and plot the mean visual context magnitude
    at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: visual context diagnostics by caption timestep
    """

    try:

        # calculate mean visual context magnitude by caption position
```

```
        visual_context_diagnostics = (
            decoder.calculate_visual_context_magnitude_by_timestep(
                data_loader,
                device
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual context magnitude by caption position
        plt.plot(
            visual_context_diagnostics["Timestep"],
            visual_context_diagnostics["Mean Visual Context Magnitude"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Context Magnitude")
        plt.title("Mean Visual Context Magnitude by Caption Position")
        plt.grid(True)

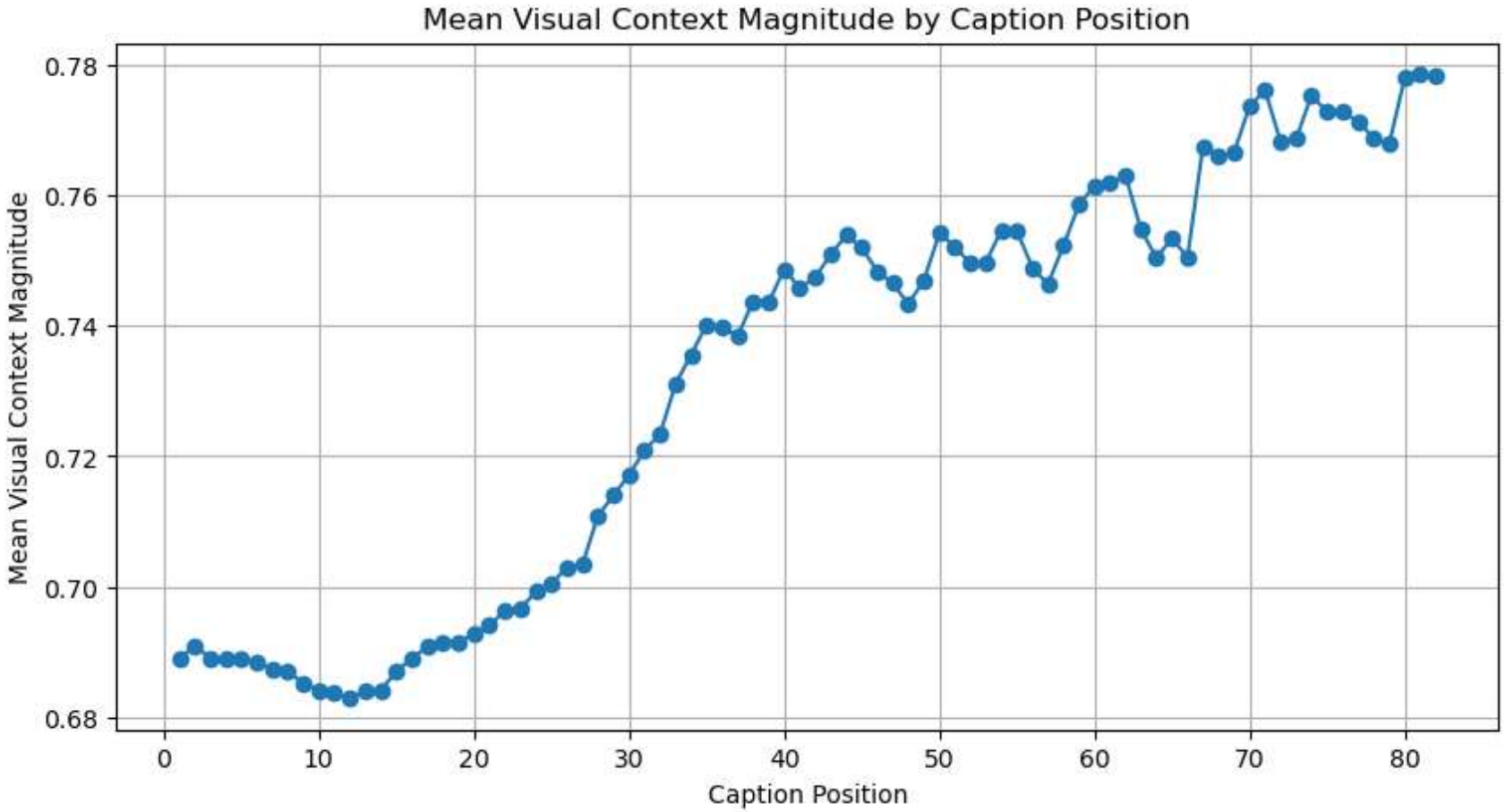
        # display plot
        plt.show()

        return visual_context_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [50]:
```
# visual context magnitude
visual_context_diagnostics = plot_visual_context_diagnostics(decoder, valid_loader, device)
```

```
Visual Context Diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [51]:
```
# display visual context magnitude
display(visual_context_diagnostics)
```

	Timestep	Mean Visual Context Magnitude	Valid Captions
0	1	0.689010	9904.0
1	2	0.690869	9903.0
2	3	0.689081	9898.0
3	4	0.688826	9826.0
4	5	0.688911	9715.0
...	...	...	...
77	78	0.768883	186.0
78	79	0.767862	177.0
79	80	0.777994	170.0
80	81	0.778591	162.0
81	82	0.778372	152.0

82 rows × 3 columns

In [52]:
```
def plot_attention_entropy_diagnostics(
    decoder,
    data_loader,
    device
):
    """
    Calculate and plot normalized attention entropy
    at each caption position.

    Args:
        decoder: trained decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: attention entropy diagnostics
        by caption timestep
```

```
    """
    try:

        # calculate attention entropy by caption position
        attention_entropy_diagnostics = (
            decoder.calculate_attention_entropy_by_timestep(
                data_loader,
                device
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot normalized attention entropy
        plt.plot(
            attention_entropy_diagnostics["Timestep"],
            attention_entropy_diagnostics[
                "Normalized Attention Entropy"
            ],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Normalized Attention Entropy")
        plt.title("Attention Entropy by Caption Position")

        # normalized entropy ranges from 0 to 1
        plt.ylim(0, 1)
        plt.grid(True)

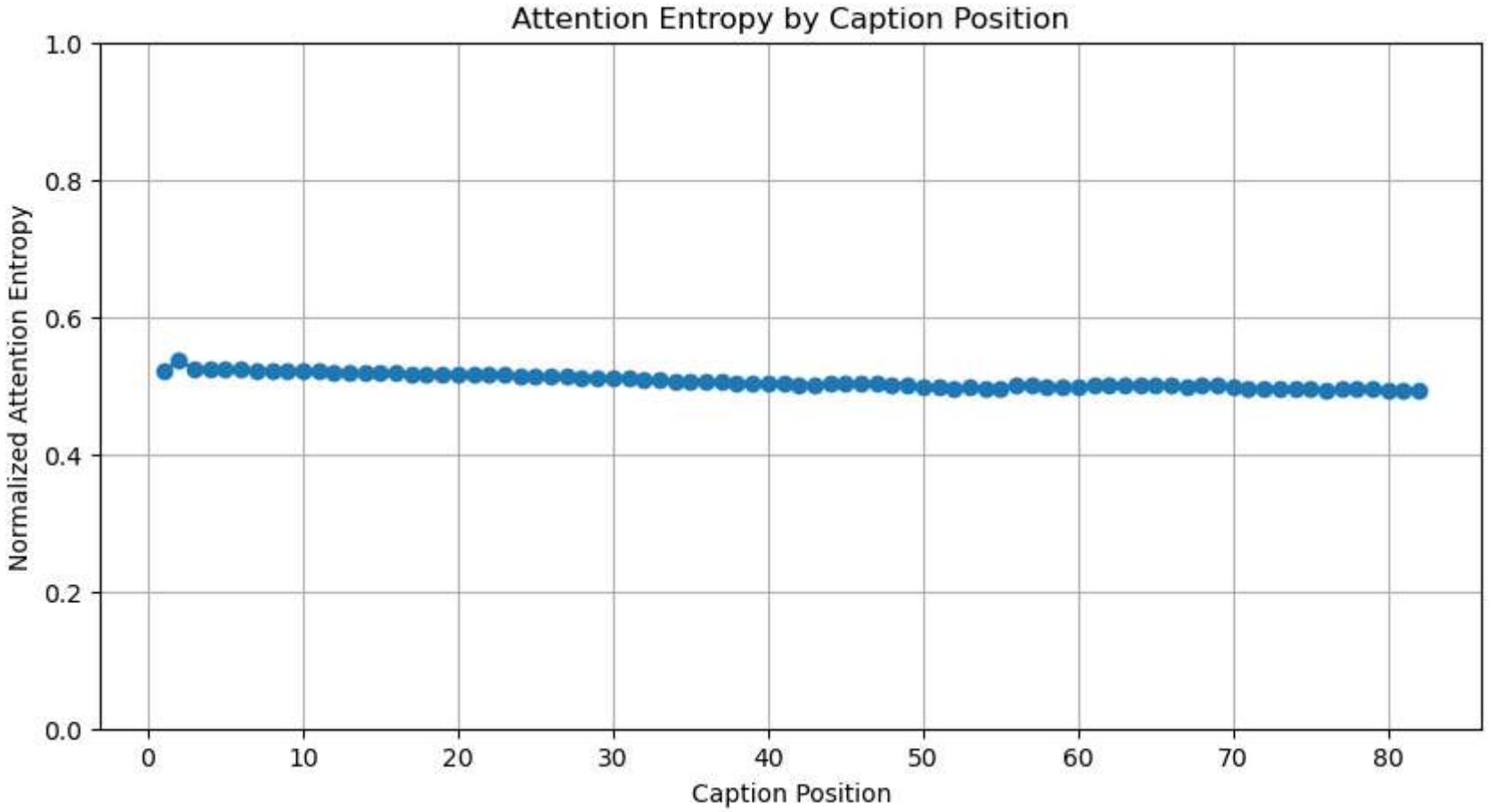
        # display plot
        plt.show()

        return attention_entropy_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [53]:
```
# attention entropy shows how concentrated or spread out the model's visual attention is across the image regions
at each caption position
# attention entropy by timestep
attention_entropy_diagnostics = plot_attention_entropy_diagnostics(decoder, valid_loader, device)
```

```
Attention entropy diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [54]:
```
# display attention entropy diagnostics
display(attention_entropy_diagnostics)
```

	Timestep	Mean Attention Entropy	Normalized Attention Entropy	Valid Captions
0	1	2.027399	0.520939	9904.0
1	2	2.085289	0.535813	9903.0
2	3	2.042206	0.524743	9898.0
3	4	2.037621	0.523565	9826.0
4	5	2.037260	0.523472	9715.0
...	...	...	...	...
77	78	1.924734	0.494559	186.0
78	79	1.926267	0.494953	177.0
79	80	1.921968	0.493848	170.0
80	81	1.917037	0.492581	162.0
81	82	1.915469	0.492178	152.0

82 rows × 4 columns

In [55]:
```
# calculate the out-of-vocabulary statistics for the validation captions against the final vocabulary
valid_oov = vocabulary.calculate_oov_statistics(valid_caption_data, dataset_name="Validation")
```

In [56]:
```
def plot_oov_statistics(oov_statistics):
    """
    Display token-level and caption-level out-of-vocabulary (OOV) statistics
    as horizontal percentage-based stacked bar charts with summary tables

    Args:
        oov_statistics (dict): OOV statistics returned by calculate_oov_statistics()
    """
    try:

        # retrieve dataset name
        dataset_name = oov_statistics["dataset"]
```

```
        # retrieve total number of tokens
        total_tokens = oov_statistics["total_tokens"]

        # retrieve total number of out-of-vocabulary tokens
        oov_tokens = oov_statistics["oov_tokens"]

        # retrieve precentage of tokens out-of-vocabulary
        token_oov_percentage = oov_statistics["oov_rate"]

        # calculate percentage of tokens that are present in the vocabulary
        in_vocabulary_percentage = 100 - token_oov_percentage

        # retrieve total number of captions
        total_captions = oov_statistics["total_captions"]

        # retrieve number of captions containing at least one out-of-vocabulary token
        captions_with_oov = oov_statistics["captions_with_oov"]

        # retrieve percentage of captions containing at least one out-of-vocabulary token
        caption_oov_percentage = oov_statistics["oov_caption_percentage"]

        # calculate percentage of captions containing no OOV words
        captions_without_oov_percentage = 100 - caption_oov_percentage

        # create vertically stacked charts
        fig, axes = plt.subplots(2, 1, figsize=(10, 5))

        # plot the percentage of tokens that are present in the vocabulary
        axes[0].barh(0, in_vocabulary_percentage, height=0.25, color="black", edgecolor="black", linewidth=0.8)

        # plot the percentage of out-of-vocabulary tokens
        axes[0].barh(0, token_oov_percentage, left=in_vocabulary_percentage, height=0.25, color="lightgrey",
edgecolor="grey", linewidth=0.8, hatch="//")

        # set token-level chart title
        axes[0].set_title(f'{dataset_name} Token-Level OOV', loc="left", pad=10)

        # display token category labels centred above each bar segment
        axes[0].text(in_vocabulary_percentage / 2, 0.22, "In Vocabulary", ha="center", va="bottom")
        axes[0].text(in_vocabulary_percentage + (token_oov_percentage / 2), 0.22, "OOV", ha="center", va="bottom")

        # create token-level summary table
```

```
        token_table = axes[0].table(cellText=[[f"{total_tokens:,}", f"{oov_tokens:,}", f"
{token_oov_percentage:.2f}%"]],
                                    colLabels=["Total Tokens", "OOV Tokens", "OOV Rate (%)"], cellLoc="center",
loc="bottom",
                                    bbox=[0, -0.42, 1, 0.30])

        # set token-level table borders
        for cell in token_table.get_celld().values():
            cell.set_linewidth(0.8)

        # plot the percentage of captions containing no OOV words
        axes[1].barh(0, captions_without_oov_percentage, height=0.25, color="black", edgecolor="black",
linewidth=0.8)

        # plot the percentage of captions containing at least one OOV word
        axes[1].barh(0, caption_oov_percentage, left=captions_without_oov_percentage, height=0.25,
                     color="lightgrey", edgecolor="grey", linewidth=0.8, hatch="//")

        # set caption-level chart title
        axes[1].set_title(f'{dataset_name} Caption-Level OOV', loc="left", pad=10)

        # display caption category labels centred above each bar segment
        axes[1].text(captions_without_oov_percentage / 2, 0.22, "Without OOV", ha="center", va="bottom")
        axes[1].text(captions_without_oov_percentage + (caption_oov_percentage / 2), 0.22, "With OOV", ha="center",
va="bottom")

        # create caption-level summary table
        caption_table = axes[1].table(cellText=[[f"{total_captions:,}", f"{captions_with_oov:,}", f"
{caption_oov_percentage:.2f}%"]],
                                      colLabels=["Total Captions", "Captions With OOV Word", "OOV Caption (%)"],
cellLoc="center", loc="bottom",
                                      bbox=[0, -0.42, 1, 0.30])

        # set caption-level table borders
        for cell in caption_table.get_celld().values():
            cell.set_linewidth(0.8)

        # remove axes, tick marks and borders
        for axis in axes:

            # set percentage scale
            axis.set_xlim(0, 100)
```

```
            # set vertical spacing
            axis.set_ylim(-0.45, 0.55)

            # remove axis tick marks
            axis.set_xticks([])
            axis.set_yticks([])

            # remove chart borders
            for spine in axis.spines.values():
                spine.set_visible(False)

        # reduce spacing between charts
        plt.subplots_adjust(hspace=0.75)

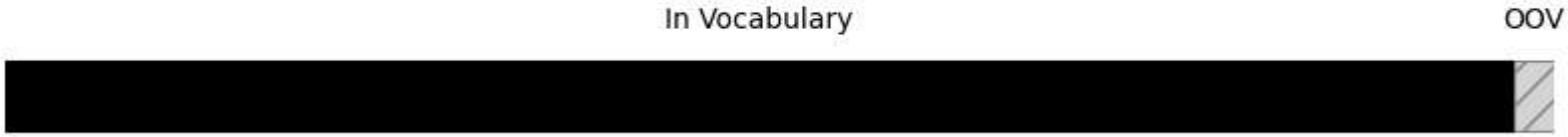
        # display charts
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [57]:

```
# display OOV statistics graphically
plot_oov_statistics(valid_oov)
```

Validation Token-Level OOV



Total Tokens	OOV Tokens	OOV Rate (%)
227,293	5,825	2.56%

Validation Caption-Level OOV



Total Captions	Captions With OOV Word	OOV Caption (%)
9,904	3,082	31.12%

In [58]:
```
def plot_caption_length_comparison(training_captions, valid_captions):
    """
    Calculate, summarise and plot training and validation caption-length statistics

    Args:
        training_captions: cleaned training Captions object
        valid_captions: cleaned validation Captions object

    Returns:
        tuple:
            training_caption_lengths: training caption lengths
            validation_caption_lengths: validation caption lengths
```

```
        caption_length_summary: summary statistics for training and validation caption lengths
    """

    try:

        # calculate training caption lengths
        training_caption_lengths = (
            training_captions.calculate_caption_lengths()
        )

        # calculate validation caption lengths
        validation_caption_lengths = (
            valid_captions.calculate_caption_lengths()
        )

        # summarise training and validation caption lengths
        caption_length_summary = pd.DataFrame([
            {
                "Dataset": "Training",
                "Captions": len(training_caption_lengths),
                "Mean Length": training_caption_lengths.mean(),
                "Median Length": training_caption_lengths.median(),
                "Minimum Length": training_caption_lengths.min(),
                "Maximum Length": training_caption_lengths.max(),
                "Std Dev": training_caption_lengths.std(),
            },
            {
                "Dataset": "Validation",
                "Captions": len(validation_caption_lengths),
                "Mean Length": validation_caption_lengths.mean(),
                "Median Length": validation_caption_lengths.median(),
                "Minimum Length": validation_caption_lengths.min(),
                "Maximum Length": validation_caption_lengths.max(),
                "Std Dev": validation_caption_lengths.std(),
            }
        ])

        # select caption-length statistics for graphical comparison
        length_statistics = (
            caption_length_summary
            .set_index("Dataset")[
                [
```

```
                    "Mean Length",
                    "Median Length",
                    "Minimum Length",
                    "Maximum Length",
                    "Std Dev"
                ]
            ]
        )

        # create grouped bar chart
        length_statistics.T.plot(
            kind="bar",
            figsize=(10, 5)
        )

        # configure plot
        plt.xlabel("Caption Length Statistic")
        plt.ylabel("Caption Length (words)")
        plt.title("Training and Validation Caption Length Comparison")
        plt.xticks(rotation=45)
        plt.legend(title="Dataset")
        plt.grid(axis="y")

        # prevent labels from overlapping
        plt.tight_layout()

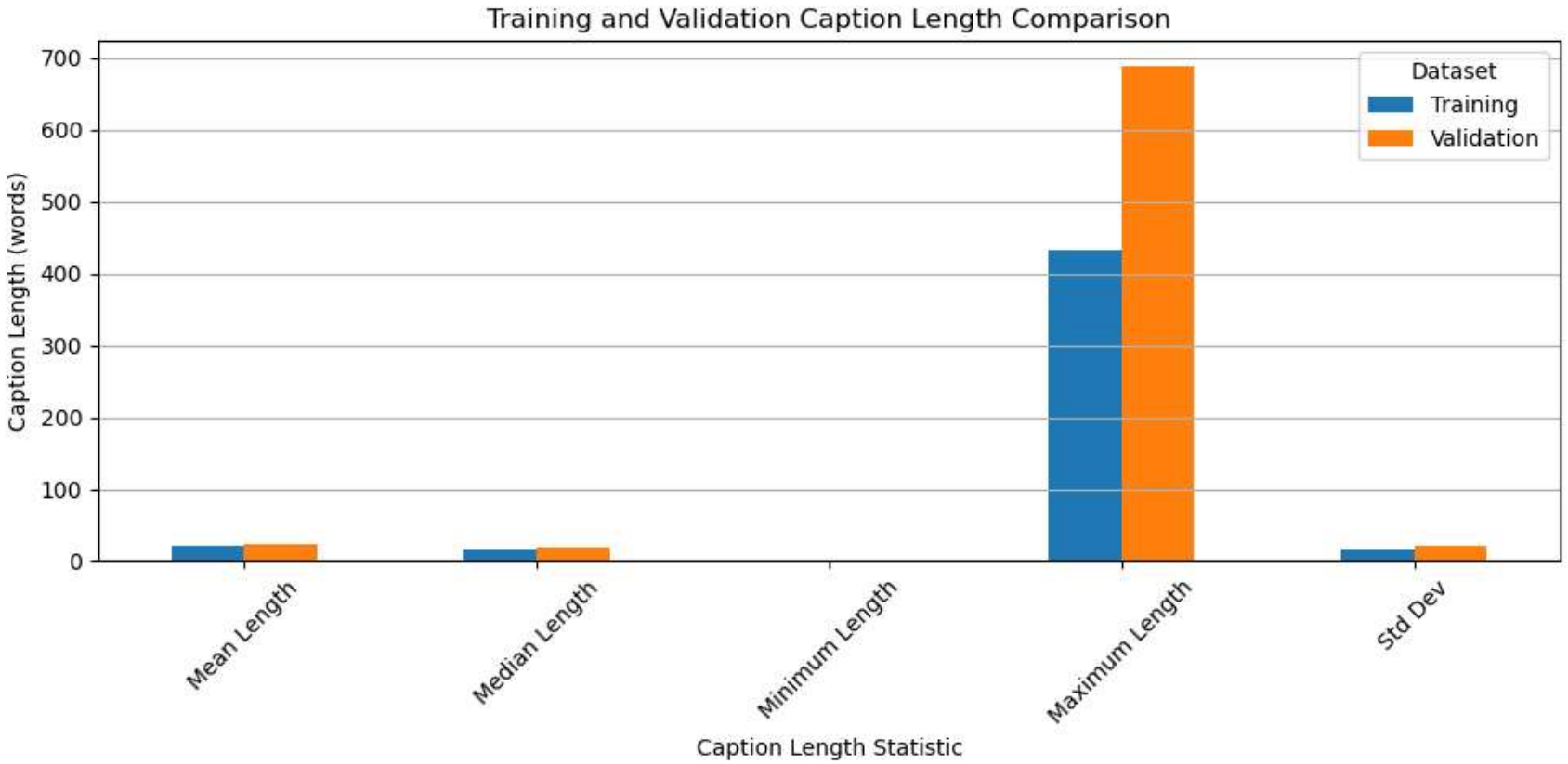
        # display plot
        plt.show()

        return (
            training_caption_lengths,
            validation_caption_lengths,
            caption_length_summary
        )

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [59]:

```
# calculate, summarise and compare training and validation caption lengths
training_caption_lengths, validation_caption_lengths, caption_length_summary =
plot_caption_length_comparison(training_captions, valid_captions)
```



In [60]:
```
# display caption length summary
display(caption_length_summary)
```

	Dataset	Captions	Mean Length	Median Length	Minimum Length	Maximum Length	Std Dev
0	Training	59958	21.378615	17.0	1	432	16.277227
1	Validation	9904	22.949616	18.0	0	689	21.833881

In [61]:
```
def plot_caption_length_loss(decoder, data_loader, device, bin_size=10):
    """
    Calculate and plot mean validation token loss

    Args:
        decoder: trained decoder model
```

```
        data_loader: validation DataLoader
        device: device used for model calculations
        bin_size (int): number of words in each caption-length range

    Returns:
        tuple:
            validation_caption_losses: individual caption-level loss results
            caption_length_loss_summary: mean loss grouped by caption-length range
    """

    try:

        # calculate individual caption losses and grouped summary
        validation_caption_losses, caption_length_loss_summary = (
            decoder.calculate_loss_by_caption_length(
                data_loader,
                device,
                bin_size=bin_size
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean token loss by caption-length range
        plt.plot(
            caption_length_loss_summary["Length Range"].astype(str),
            caption_length_loss_summary["Mean_Token_Loss"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Length (words)")
        plt.ylabel("Mean Token Loss")
        plt.title("Validation Loss by Caption Length")
        plt.xticks(rotation=45)
        plt.grid(True)

        # prevent labels from overlapping
        plt.tight_layout()

        # display plot
```

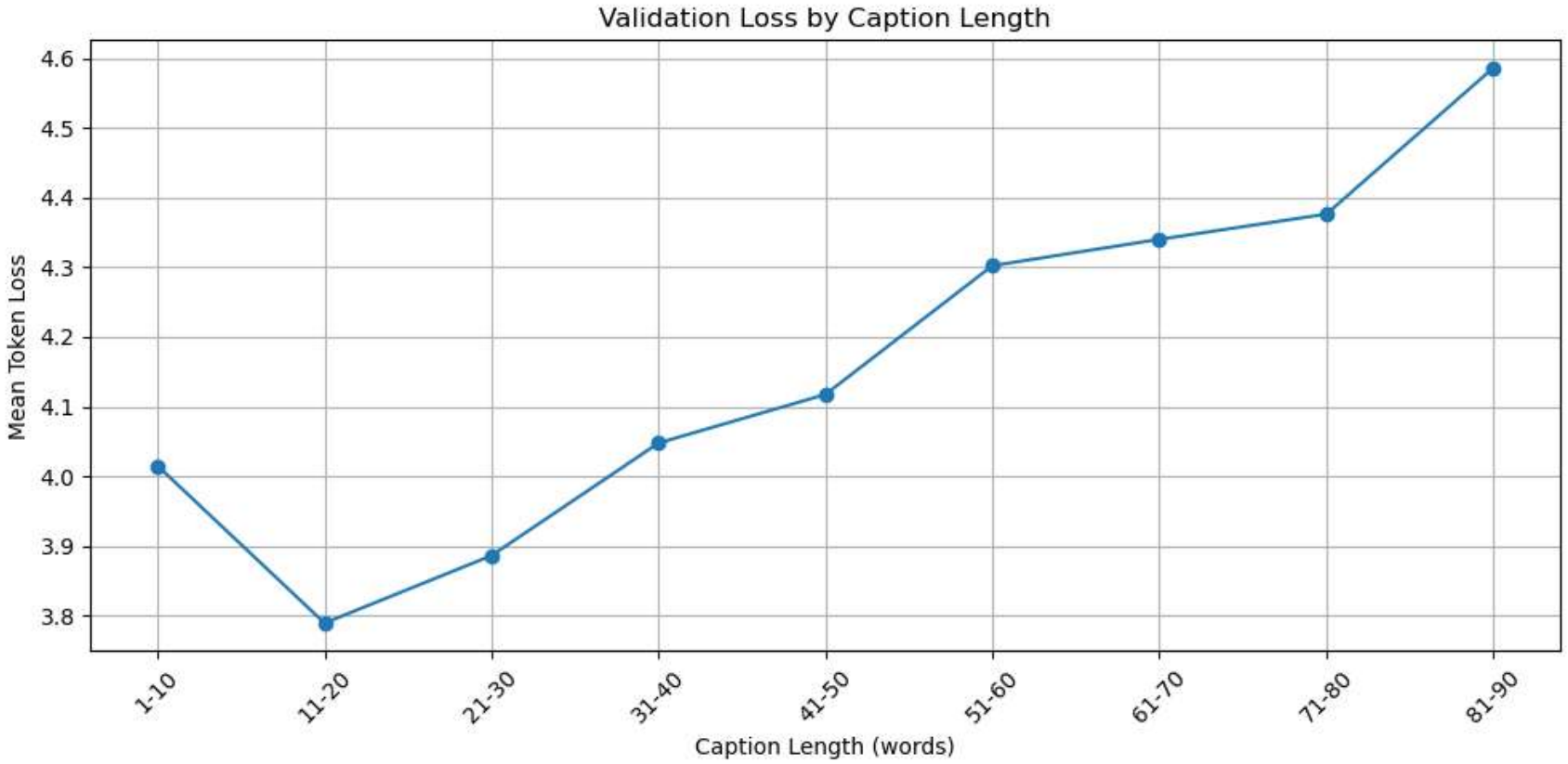
```
        plt.show()

        return (validation_caption_losses, caption_length_loss_summary)

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [62]:
```
# calculate individual validation-caption loss statistics
validation_caption_losses, caption_length_loss_summary = (plot_caption_length_loss(decoder, valid_loader, device,
bin_size=10))
```

```
Loss By Caption Length:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [63]:
```
# display vaidation caption losses
display(validation_caption_losses)
```

	Caption Length	Mean Token Loss	Total Caption Loss	Length Range
0	17	2.447544	44.055790	11-20
1	45	3.997548	183.887192	41-50
2	81	5.244384	430.039490	81-90
3	6	3.216665	22.516655	1-10
4	4	4.263503	21.317514	1-10
...	...	...	...	...
9899	22	3.850857	88.569702	21-30
9900	23	2.263672	54.328121	21-30
9901	44	3.632492	163.462158	41-50
9902	6	6.756030	47.292206	1-10
9903	80	5.205110	421.613892	71-80

9904 rows × 4 columns

In [64]:
```
# display caption lenght loss summary
display(caption_length_loss_summary)
```

	Length Range	Captions	Mean_Caption_Length	Mean_Token_Loss	Mean_Total_Caption_Loss
0	1-10	2115	7.134279	4.015136	32.326084
1	11-20	3570	15.245938	3.789309	61.527458
2	21-30	2077	24.937891	3.886162	100.965141
3	31-40	1022	35.006849	4.047737	145.847936
4	41-50	490	45.087755	4.117484	189.835723
5	51-60	259	55.355212	4.302351	242.457622
6	61-70	139	65.366906	4.340162	288.232398
7	71-80	80	75.487500	4.376307	334.945799
8	81-90	152	81.000000	4.586187	376.067327

References

Bahdanau, D., Cho, K. and Bengio, Y., Neural Machine Translation by Jointly Learning to Align and Translate (2015) Available At : https://arxiv.org/abs/1409.0473

Cruz Diaz, N.P. and Mana Lopez, M, 'An analysis of biomedical tokenization: problems and strategies', Proceedings of the Sixth International Workshop on Health Text Mining and Information Analysis', pages 40-49 (2015). Available At: https://aclanthology.org/W15-2605/

He, B., Guan, Y., Cheng, J., Cen, K. and Hua, W. CRFs Based De-identification Of Medical Records. Science Direct, 2015. Available at: https://doi.org/10.1016/j.jbi.2015.08.012

Moon, S.R., Pakhomov, S., Ryan, J. and Melton, G.B., 'Automated non-alphanumeric symbol resolution in clinical texts', AMIA Annual Symposium Proceedings (2011) pages 979–986. Available At : https://pmc.ncbi.nlm.nih.gov/articles/PMC3243158/

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

Python Software Foundation, re — Regular expression operations (2026). Available At: https://docs.python.org/3/library/re.html

Robertson, S. (2024) NLP From Scratch: Translation with a Sequence to Sequence Network and Attention. PyTorch Tutorials. Available At: https://docs.pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset. Available At : https://zenodo.org/records/10821435

sgrvinod, A PyTorch Tutorial to Image Captioning: Show, Attend, and Tell, GitHub (2026). Available At : https://github.com/sgrvinod/a-PyTorch-Tutorial-to-Image-Captioning

Xu, K.et al, Show, Attend and Tell: Neural Image Caption Generation with Visual Attention (2015). Available At :https://proceedings.mlr.press/v37/xuc15.html